

F21SC Coursework 1

By: Mohamad Ajaz

H00458396

Introduction:

The purpose of this report is to provide a simple to follow guide on setting up Squash Browser.

Squash Browser is a simple and intuitive new way to browse on the web. It isn't just another chromium web client; it is a simple, intuitive, and radically new way to experience the web!

It rivals already existing heavy and clunky chromium web browsers such as Google Chrome, Opera, Edge, etc. Here at squash labs we revolutionized the way browsers work by not even including a web rendering engine! Crazy right? Not only does this make our browser the best performative browser in the market, there is absolutely no way for third-party trackers, pixels, or intrusive JavaScript to execute or be displayed in our application, meaning you get true, undeniable, and effortless privacy!

Squash Browser is not a bug, it's a feature. We're offering a direct, powerful, and text-only browsing experience.

The way the internet was always meant to be.

Assumptions:

- The user history list would not need to show the title for each history item.
- A list was used as the data structure for the navigation stack as it allows forward and backward access without affecting the stack.
- The initial home page URL would initialize with "https://hw.ac.uk", but the user would be allowed to change it to any URL as they please. ("https://hw.ac.uk" is just a suggestion for the user at the start of the browser)
- Navigation stack wouldn't need to be saved in the database as it was not specified in the requirements. As such when the user loads the browser again only the the most recent history would be loaded.
- Using the alternative Avalonia was allowed by my professor, while mentioned in the spec that GTK# Library for linux must be used.
- The user would be able to add any amount of the same bookmark URL

Feature checklist:

FEATURE	STATUS
Sending HTTP requests and displaying the contents of the response	Fully Implemented
Displaying the HTTP response status code	Fully Implemented
Displaying the web page's title	Fully Implemented
Reload the current web page	Fully Implemented
Setting the home page	Fully Implemented

Adding pages to bookmarks list	Fully Implemented
Naming bookmarks	Fully Implemented
Deleting bookmarks	Fully Implemented
Navigating to bookmarks	Fully Implemented
Maintaining history	Fully Implemented
Navigating to previous/next page in history	Fully Implemented
Navigating to a selected page in history	Fully Implemented
List five urls from the web-page in the side panel	Fully Implemented
Overall good GUI experience	Subjective of the User :)
Settings (home page, bookmarks, history) persist across sessions	Mostly implemented - As under my assumptions i assumed navigation history would not need to be saved.
(Advanced) Database used as storage	Fully Implemented
(Advanced) Added multi-user functionality.	Not Implemented

Design Considerations:

1. Architecture:

The architecture used in this project is the MVVM (Model, View, ViewModel). MVVM basically allows us to separate the GUI logic with the business/backend logic, effectively allowing me/devs to maintain and scale the codebase in an easier fashion. It is also regarded as the go-to default architecture for many frameworks like WPF, Xamarin, etc. hence another reason to use it.

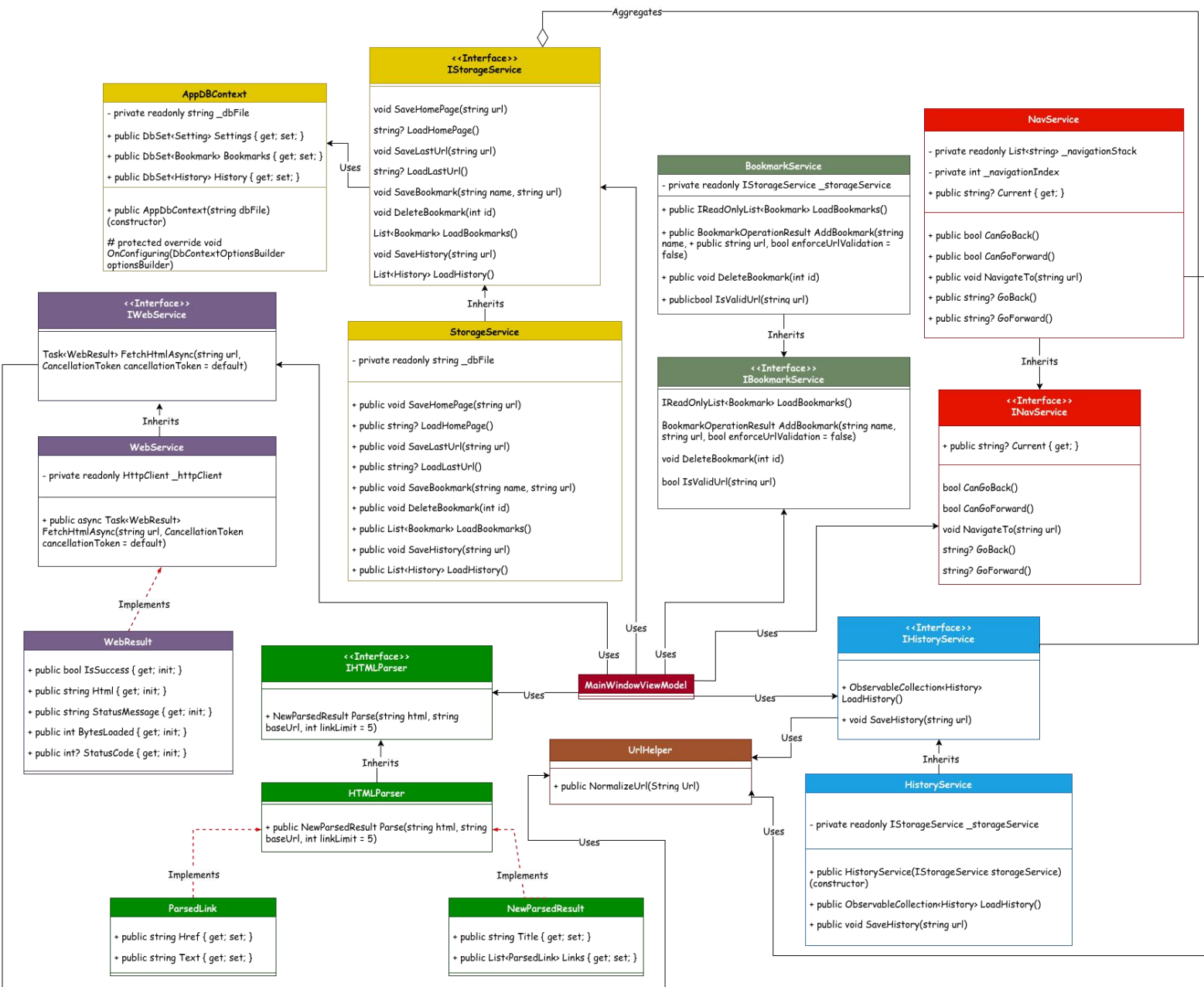
Below is the file structure of the squash web browser project. Commands folder holds class that acts as the binder of the UI elements to the ViewModel logic. Controls folder holds custom reusable UI elements such as the address bar, bookmarks panel, etc. Models folder holds application's data classes. These classes represent the core objects of the application, such as a Bookmark or a History entry. Services holds the classes that provide core functionality for the app such as IWebService to make http requests add IStorageService that handles all the database logic. ViewModels folder

holds the core business logic binding UI elements to specific functions from the Services folder to be called. Finally views folder contains the main windows and pages UI of the application's user interface.

```
Squash_Web_Browser/
├── Commands/
│   └── RelayCommand.cs
├── Controls/
│   ├── AddressBar.axaml
│   ├── AddressBar.axaml.cs
│   ├── BookmarksPanel.axaml
│   ├── BookmarksPanel.axaml.cs
│   ├── EditHomePagePanel.axaml
│   ├── EditHomePagePanel.axaml.cs
│   ├── HistoryPanel.axaml
│   └── HistoryPanel.axaml.cs
├── Models/
│   ├── Bookmark.cs
│   ├── History.cs
│   └── Setting.cs
├── Services/
│   ├── AppDbContext.cs
│   ├── UrlHelper.cs
│   ├── IBookmarkService.cs
│   ├── IHistoryService.cs
│   ├── IHtmlParser.cs
│   ├── INavService.cs
│   ├── IStorageService.cs
│   └── IWebService.cs
├── ViewModels/
│   ├── MainWindowViewModel.cs
│   └── ViewModelBase.cs
├── Views/
│   ├── App.axaml
│   ├── App.axaml.cs
│   ├── MainWindow.axaml
│   └── MainWindow.axaml.cs
└── browserdata.db
```

Appendix 1 - Class diagrams:

This section covers the class diagram of the services folder, containing classes that handle most of the business logic such as the html parsing, database storing, url navigation, bookmark, history, etc.



Entity Relationship Diagram (Advanced functionality 1):

Settings	
PK	<u>Key - String</u>
	Value - String

History	
PK	<u>ID - Int (Auto-increment)</u>
	Url - String
	TimeStamp- Datetime

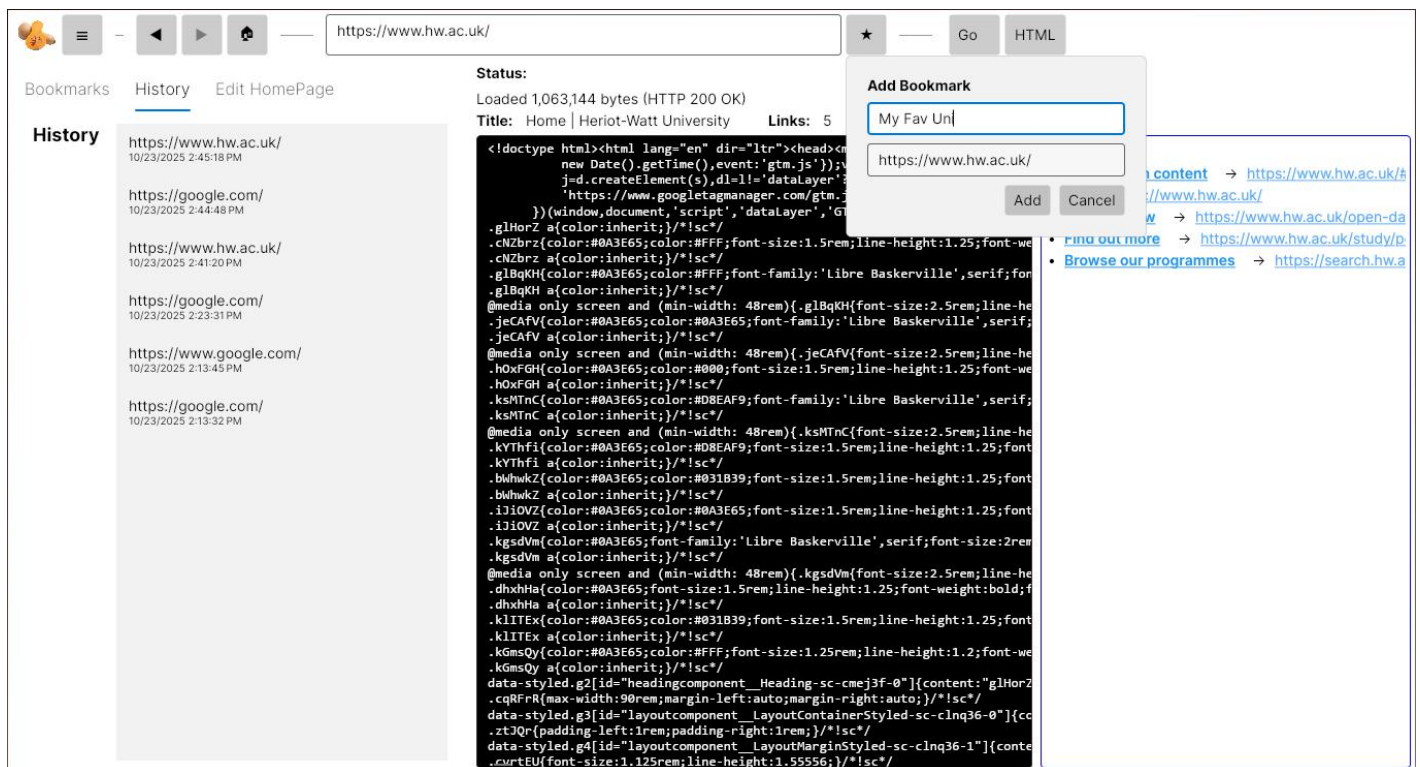
Bookmarks	
PK	<u>ID - Int (Auto-increment)</u>
	Name
	Url

The current approach to the database was to use a unique table for different user requirements. Basically since this implementation was without the multi user feature, all data is stored independently in the tables without the need for any user identification. If I were to add the multi user feature, each entity (Setting, Bookmark, History) would need an additional field UserID as the foreign key to associate the records with specific users.

Throughout the implementation however LINQ was used for all database operations such as querying, storing, updating, etc. The implementation for this code can be accessed via the IStorageService and the AppDbContext class.

GUI Design & Advanced language constructs:

The GUI is built in a way to mimic a classic web browser mostly taking inspiration from google chrome. Below is a screenshot of how the browser GUI looks like:



The top address bar hosts a sidebar toggle, Back, Forward, Reload, and Home buttons, a URL/search TextBox with Enter to navigate, a star button that opens a small dialog box to add a bookmark (name + read-only URL with Add/Cancel), a Go button, and a toggle to show/hide raw HTML. The bookmarks sidebar/panel lists saved bookmarks and supports clicking to navigate as well as add/edit/delete via simple inputs and validation. The History tab lists the most recent URLs visited by showing its URL and its timestamp accessed. Each history item can be accessed by clicking on the item. Lastly the Edit

Home Page tab allows the user to edit the home page URL. First initialized with "https://www.hw.ac.uk/" as per the spec. The user is able to go back to the home URL via the home icon button.

Developer Guide:

1. Setup Guide:

Prerequisites:

Before you begin, ensure you have the following installed:

- .NET 9.0 SDK
- VsCode, Visual Studio or any Avalonia compatible IDE/Text Editor
- Preferably a linux machine, windows untested

Getting Started:

1. I assume you will have the project zip file I uploaded from canvas.
2. Next, restore the dependencies via command: **"dotnet restore"**
3. Run the application via: **"dotnet run"**

Building An executable:

The following project can be built into a windows executable via linux command:

"dotnet publish -c Release -r win-x64 --self-contained true /p:PublishSingleFile=true"

Then, the following executable file can be found in:

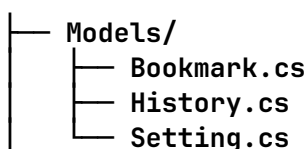
"bin/Release/net9.0/win-x64/publish/Squash_Web_Browser.exe"

This project uses the following main packages:

- Avalonia - A cross-platform UI framework for .NET.
- CommunityToolkit.Mvvm - Modern, fast, and flexible MVVM library.
- HtmlAgilityPack - Solid HTML parser that parses and grabs Tags for Title or Attributes.
- Microsoft.Data.Sqlite - SQLite database provider for Entity Framework Core.
- Microsoft.EntityFrameworkCore.Sqlite - SQLite provider for EF Core.
- LINQ - Used extensively for querying in-memory collections and database entities.
It is a part of the .NET platform and does not require a separate package.

2. Class organization and responsibilities:

As mentioned in the architecture section, the project follows a MVVM architecture to allow for clear separation of concerns:



Models: These hold the Bookmark, Settings, History entity classes which are representing the database tables holding properties like ID, Name, URL, etc.

```
├── ViewModels/
│   ├── MainWindowViewModel.cs
│   └── ViewModelBase.cs
```

ViewModel: The view model `MainWindowViewModel` handles all the application states and user interactions, it exposes:

- **Properties:** Address, HomePageUrl, HtmlSource, StatusMessage, Bookmarks, History, ParsedLinks (all `ObservableCollections` for reactive UI updates)
 - **Commands:** `FetchHtmlCommand`, `BackButtonCommand`, `ForwardButtonCommand`, `HomeButtonCommand`, `ToggleHtmlCommand`, `ShowBookmarkFlyoutCommand`, `HistoryItemClickCommand`
 - **Methods:** `FetchHtmlAsync()`, `ParseHtml()`, `GoHomeAsync()`, `GoBackAsync()`, `GoForwardAsync()`
-

```
├── Services/
│   ├── AppDbContext.cs
│   ├── UrlHelper.cs
│   ├── IBookmarkService.cs
│   ├── IHistoryService.cs
│   ├── IHtmlParser.cs
│   ├── INavService.cs
│   ├── IStorageService.cs
│   └── IWebService.cs
```

Services: These files hold most of the business logic (Http requests, database operations, navigation)

- `IWebService / WebService` - Handles HTTP communication via the `FetchHtmlAsync` method, allowing for fetching HTML content from URLs.
- `IStorageService / StorageService` - Manages persistent storage operations (save/load home page, bookmarks, history, last URL)
- `IHtmlParserService / HtmlParserService` - Parses HTML to extract titles and links
- `INavService / NavService` - Manages navigation history, the data structure used for the Navigation of history is via a `List`. As this allows forward and backward navigation without altering the stack.
- `IHistoryService / HistoryService` - Handles browsing history operations
- `IBookmarkService / BookmarkService` - Handles adding, editing, and deleting bookmark operations
- `AppDbContext` - Entity Framework Core `DbContext` managing database schema and connections
- `UrlHelper` - Standardizes URLs by ensuring they have the correct scheme (`http/https`) and formatting

```
└─ Views/
    └─ App.axaml
    └─ App.axaml.cs
    └─ MainWindow.axaml
    └─ MainWindow.axaml.cs
```

Views:

- MainWindow.axaml.cs - Main application window, wires up DataContext to MainWindowViewModel
 - App.axaml.cs - Application entry point and lifecycle management
-

```
└─ Controls/
    └─ AddressBar.axaml
    └─ AddressBar.axaml.cs
    └─ BookmarksPanel.axaml
    └─ BookmarksPanel.axaml.cs
    └─ EditHomePagePanel.axaml
    └─ EditHomePagePanel.axaml.cs
    └─ HistoryPanel.axaml
    └─ HistoryPanel.axaml.cs
```

Controls :

- AddressBar.axaml - Top navigation bar with buttons, URL input, and bookmark flyout
- BookmarksPanel.axaml - Sidebar UI for viewing bookmarks panel
- EditHomePagePanel.axaml - Sidebar UI for viewing EditHomePage panel
- HistoryPanel.axaml - Sidebar UI for viewing History panel

3. HTTP Communication Technology

HttpClient library is used for all HTTP operations. The WebService class creates HttpClient instances with a 30-second timeout and executes GetStringAsync() to fetch HTML content. Error handling manages network failures, timeouts, and invalid responses, returning appropriate status messages to the ViewModel.

4. Persistence Technology

Entity Framework Core with SQLite is used to provide persistent storage. A single database file called browserdata.db holds all the tables as mentioned in the entity relation diagram. A constructor also calls db.Database.EnsureCreated() to initialize the schema on first run. For a dev to create a new table, they must define a new DbSet<T> properties in AppDbContext class, then define their LINQ queries in the StorageService class

5. GUI Technology

As mentioned before, the Avalonia UI is responsible for rendering the interface. To know more about how to design the user interface I used the documentation [<https://docs.avaloniaui.net/docs/basics/>] and several youtube videos.

Conclusions:

The squash web browser was a fun but time consuming project; to use a new framework like Avalonia instead of Windows Forms was a hard but necessary decision to make due to my limitations as a linux only user. However said and done building an actual functioning and responsive UI is probably my most proud achievement in this project. If I were to improve on this project I would start with completing the final advanced feature that is multi user access. However due to time constraints and my limited knowledge it was not possible, as such I will leave it to my future self in another time to hopefully make this a real and full fledged browser.